Nonparametric MLR - Lab Activity - Example Solution

B.A. Hartlaub and A.S. Wagaman

Nonparametric Regression Examples

Example 1 (includes parametric regression comparison)

We use a data set on painted turtles, where we know their biological sex and their height/length/width measurements.

```
turtle <-
  read.table("https://awagaman.people.amherst.edu/nonparametricstats/paintedturtle.txt",
      header = TRUE)
```

Suppose we want to try to predict the height of the turtles, using length and sex.

Fit a nonparametric regression predicting height using length. Use *theil* for exact results since this is an SLR. Do your results make sense?

SOLUTION:

```
# yikes this doesn't run!
# with(turtle, theil(length, height, beta.0 = 0, type = "t"))
# try without the plot
with(turtle, theil(length, height, beta.0 = 0, type = "t", doplot = F))
```

We get output with the second command (the first was apparently interrupted because of issues plotting), however, the `beta.hat` and `alpha.hat` estimates are both NA. We got a CI for beta though, and there's clearly a significant relationship because `C.bar` is 0.829 and the corresponding p-value is rounded to 0.

You probably noticed something odd going on. There are many ties here among the length values. It appears that these are causing issues. If we want the nonparametric estimates, we'll have to get them ourselves.

How would we do that? A little brute code (which can likely be done more elegantly). Just follow along with what is being done here.

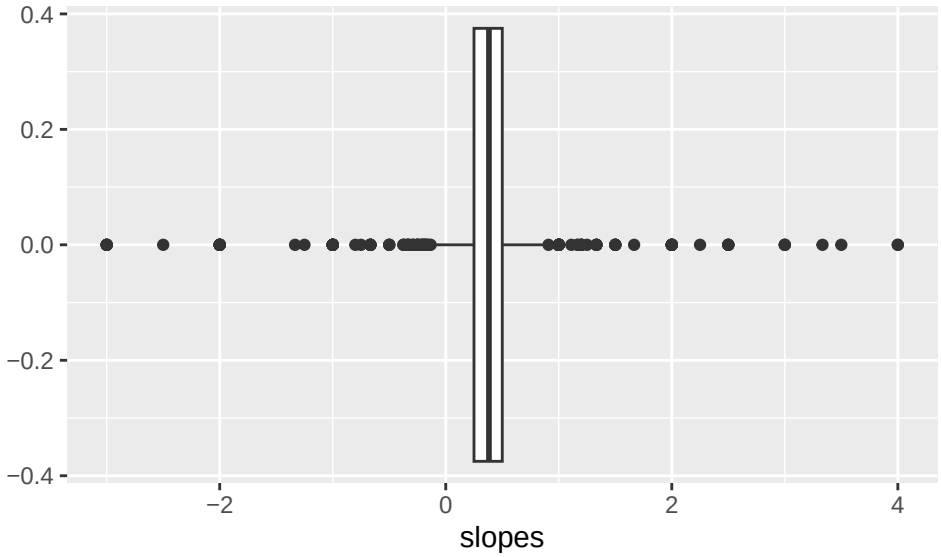
```
num <- nrow(turtle)
slopes <- NULL
for(i in 1:(num-1)){
  for(j in (i+1):num){
     slopes <- c(slopes, as.numeric(lm(height ~ length,
                                       data = turtle[c(i,j),])$coeff[2]))
  }
}
favstats(~ slopes)
```

```
##  min   Q1    median  Q3 max      mean        sd    n missing
##   -3 0.25 0.3823529 0.5   4 0.3667024 0.5360038 1117      11
```

```
gf_boxplot(~ slopes)
```

```
## Warning: Removed 11 rows containing non-finite outside the scale range
```

```
## (`stat_boxplot()`).
```



This is an approach where we iterate and each time, add a possible slope to our slopes vector. We use the parametric lm function - you could compute this with the respective X and Y's too. Why do we pull the second entry from the coefficients?

SOLUTION:

In an lm, the first coefficient is the intercept. The second is the slope coefficient for the first term. Since this is SLR, that's the one we want.

What would you report as the nonparametric slope estimate?

SOLUTION:

We saved the possible slopes into a vector, and we want the median of the possible slopes as our estimate. Thus, we'd report the median of 0.382ish as our slope estimate (depending on where you round).

We could then get the alpha (intercept estimate) fairly easily as well.

```
alphas <- with(turtle, height - 0.382*length)
median(alphas)
```

```
## [1] -1.419
```

The nonparametric regression line thus obtained manually would be:

SOLUTION:

$$PredictedMedianHeight = -1.419 + 0.382 * length$$

Compare your results to those from `rfit` rank-based approach for the SLR. How much do the lines differ?

SOLUTION:

```
myht <- rfit(height ~ length, data = turtle)
summary(myht)
```

```
## Call:
## rfit.default(formula = height ~ length, data = turtle)
##
```

```
## Coefficients:
##              Estimate Std. Error t.value p.value
## (Intercept) -2.434783   2.243433 -1.0853  0.2834
## length       0.391304   0.017464 22.4067  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.8315032
## Reduction in Dispersion Test: 227.0022 p-value: 0
```

The rfit approach yields a predicted line of:

$$PredictedMedianHeight = -2.435 + 0.391 * length.$$

The slopes are very similar, as we obtained 0.382 and 0.391, while the intercepts differ a little bit more.

Now we want to extend to multiple linear regression.

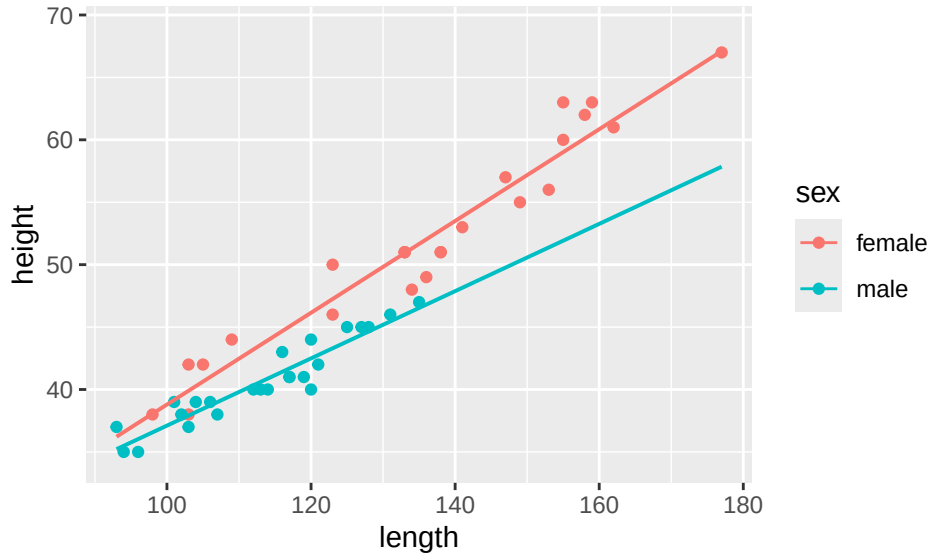
Use `sen.adichie` to test whether or not the slopes of the regression line for predicting height using length would be equal or different for the two sexes. Remember a particular data format is needed for this procedure.

SOLUTION:

To begin, although this was not requested, we'll plot this to use our intuition about what should happen. You can add the parametric fitted lines or not.

```
ggplot(turtle, aes(x = length, y = height, color = sex)) +
  geom_point() + geom_lm() # assumes slopes can differ
```

```
## Warning: Using the `size` aesthetic with geom_line was deprecated in ggplot2 3.4.0.
## i Please use the `linewidth` aesthetic instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



To run `sen.adichie`, we have to get the data in the correct format - separate data files for each group, with only the x and y (and in order, first x, then y), compiled and put into a list.

```r
mturtle <- filter(turtle, sex == "male") %>%
  dplyr::select(length, height)
fturtle <- filter(turtle, sex == "female") %>%
  dplyr::select(length, height)
z <- list(mturtle, fturtle)
sen.adichie(z)
```

```
##
## Null: all slopes are equal
## V = 9.794, P = 0.002
```

Here, our V is 9.794 and the corresponding p-value is 0.002. We would reject the null hypothesis at the commonly used significance levels. This indicates that we need non-parallel slopes. That is, a model where the groups should be able to have different slopes would be better.

Try `rfit` with a categorical predictor and interaction term to do the same test as `sen.adichie`. Are the results consistent?

SOLUTION:

```r
# remember if K > 2, this means fitting two models, then using drop.test
# if K = 2, there is another option

modint <- rfit(height ~ length*sex, data = turtle)
modmain <- rfit(height ~ length + sex, data = turtle)

# first approach, fit interaction model, check single interaction slope
summary(modint)
```

```
## Call:
## rfit.default(formula = height ~ length * sex, data = turtle)
##
## Coefficients:
##                 Estimate Std. Error t.value p.value
## (Intercept)     2.636360   2.375994  1.1096 0.27321
## length          0.363636   0.017219 21.1187 < 2e-16 ***
## sexmale         6.980967   4.259569  1.6389 0.10837
## length:sexmale -0.087774   0.035513 -2.4716 0.01739 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.8900561
## Reduction in Dispersion Test: 118.7347 p-value: 0
```

```r
# second approach, use drop.test to compare models
drop.test(modint, modmain)
```

```
##
## Drop in Dispersion Test
## F-Statistic     p-value
##   7.7499959   0.0078878
```

In the first approach (which relies on K being 2), we fit a model with an interaction term between length and sex to see if we have the same results as `sen.adichie`. If the interaction term is significantly different from 0, then the two groups require different slopes for length. We can see the p-value for the interaction term is 0.01739, which is well below 0.05. Thus, we reach the same conclusion as `sen.adichie` - we should not use parallel slopes here.

In the second approach, we use a `drop.test()` to compare nonparametric models - you compare the model with and without the interaction. The p-value is again small (0.0079), so we reach the same conclusion with this approach.

Try `lm` following the same approach as above. Are the nonparametric results about whether you need different slopes for the sexes consistent with the parametric results?

SOLUTION:

```
# remember if K > 2, this means fitting two models, then using anova
# if K = 2, there is another option

paramodint <- lm(height ~ length * sex, data = turtle)
paramodmain <- lm(height ~ length + sex, data = turtle)

# first approach, fit interaction model, check single interaction slope
msummary(paramodint)
```

```
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)     2.03999    2.16877   0.941  0.35204
## length          0.36755    0.01576  23.323  < 2e-16 ***
## sexmale         8.13218    3.89842   2.086  0.04281 *
## length:sexmale -0.09821    0.03250  -3.022  0.00418 **
##
## Residual standard error: 1.606 on 44 degrees of freedom
## Multiple R-squared:  0.9655, Adjusted R-squared:  0.9631
## F-statistic: 410.5 on 3 and 44 DF,  p-value: < 2.2e-16
```

```
# second approach, use anova to compare models, swap order!
anova(paramodmain, paramodint)
```

```
## Analysis of Variance Table
##
## Model 1: height ~ length + sex
## Model 2: height ~ length * sex
##   Res.Df    RSS Df Sum of Sq      F   Pr(>F)
## 1     45 137.02
## 2     44 113.48  1    23.547 9.1301 0.004179 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Under either approach, we see tiny p-values for the interaction coefficient / corresponding test. Thus, the parametric approach agrees with the nonparametric ones. We should have different slopes for the sexes here for length when trying to predict height.

Example 2

```
primate <-
  read.table("https://awagaman.people.amherst.edu/nonparametricstats/primatedata.txt",
                      header = TRUE)
```

The data set *primatedata.txt* contains variables about shoulder blades from 105 primates. There are 5 indices (basically measured distances): AD.BD, AD.CD, EA.CD, Dx.CD, and SH.ACR, and 2 angles recorded (EAD and beta) for each shoulder blade. The primate genus (plural: genera) is also recorded, as the variable *class*. Gibbons (Hylobates), orangutangs (Pongo), chimpanzees (Pan), gorillas (Gorilla), and man (Homo) are all represented in the data set.

Pick three of the five genera to focus on. Which three do you think it would be interesting to explore differences between?

SOLUTION:

Any three genera can be picked. Later answers may differ based on the three genera you picked.

For the purpose of this solution, we decided to pick Homo, Pan, and Pongo.

Create a data set with only your three genera present. It may help to add a mutate where you reset the factor levels (just do: `class = factor(class)`) so it understands there are only 3 levels.

SOLUTION:

```
mythree <- filter(primate, class == "Pan" | class == "Pongo" | class == "Homo") %>%
  mutate(class = factor(class))
```

Use `sen.adichie` to test whether or not the slopes of the regression line for predicting AD.BD using AD.CD would be equal or different for your choice of three genera.

SOLUTION:

```
Homo <- filter(mythree, class == "Homo") %>%
  dplyr::select(AD.CD, AD.BD)
Pan <- filter(mythree, class == "Pan") %>%
  dplyr::select(AD.CD, AD.BD)
Pongo <- filter(mythree, class == "Pongo") %>%
  dplyr::select(AD.CD, AD.BD)
z <- list(Homo, Pan, Pongo)
sen.adichie(z)
```

```
##
## Null: all slopes are equal
## V = 21.43, P = 0
```

Our test statistic V is 21.43 and the p-value is reported as 0. Based on our evidence, the slopes do not appear to be equal for the group of three genera selected. Thus, we would want to use an interaction term with class.

Use `rfit` to test whether or not you can predict *EAD* using all 5 indices (AD.BD, AD.CD, EA.CD, Dx.CD, and SH.ACR) with each (separately) interacted with genera (which includes genera's main effect), versus a null model.

SOLUTION:

This is intended to be on your subset of three genera (so the output isn't as long).

```
eadmodfull <- rfit(EAD ~ AD.BD*class + AD.CD*class + EA.CD*class +
                     Dx.CD*class + SH.ACR*class, data = mythree)
summary(eadmodfull, overall.test = 'drop') # summary(eadmodfull) works too
```

```
## Call:
## rfit.default(formula = EAD ~ AD.BD * class + AD.CD * class +
##     EA.CD * class + Dx.CD * class + SH.ACR * class, data = mythree)
##
## Coefficients:
##                     Estimate Std. Error  t.value   p.value
## (Intercept)       184.806009  18.292150  10.1030 2.579e-14 ***
## AD.BD              -1.366880   0.225643  -6.0577 1.157e-07 ***
## classPan            5.486631  43.242840   0.1269  0.899482
## classPongo         41.344087  38.008209   1.0878  0.281276
## AD.CD               0.419113   0.175985   2.3815  0.020603 *
```

```
## EA.CD              -1.159177   0.113730 -10.1923 1.862e-14 ***
## Dx.CD               3.658282   1.229473   2.9755  0.004285 **
## SH.ACR              0.131778   0.087453   1.5068  0.137374
## AD.BD:classPan      0.266383   0.348313   0.7648  0.447556
## AD.BD:classPongo    0.064599   0.500320   0.1291  0.897721
## classPan:AD.CD     -0.374557   0.193529  -1.9354  0.057907 .
## classPongo:AD.CD   -0.414161   0.300391  -1.3787  0.173363
## classPan:EA.CD      0.474009   0.248523   1.9073  0.061524 .
## classPongo:EA.CD   -0.218270   0.254497  -0.8577  0.394675
## classPan:Dx.CD     -1.020050   2.497598  -0.4084  0.684501
## classPongo:Dx.CD   -0.265637   2.324121  -0.1143  0.909405
## classPan:SH.ACR    -0.244111   0.187685  -1.3006  0.198614
## classPongo:SH.ACR   0.013750   0.215316   0.0639  0.949306
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.8323713
## Reduction in Dispersion Test: 16.64925 p-value: 0
```

The test statistic is 16.65 with a corresponding p-value of basically 0. There is evidence at least one term in the model (and there are many!) is useful for predicting EAD.

Use `rfit` to test whether or not you can drop ALL the interactions from the model above and still predict *EAD* with the 5 indices and genera as main effects.

SOLUTION:

```
eadmodreduced <- rfit(EAD ~ AD.BD + AD.CD + EA.CD + Dx.CD + SH.ACR + class,
                      data = mythree)
drop.test(eadmodfull, eadmodreduced) # larger model first
```

```
##
## Drop in Dispersion Test
## F-Statistic     p-value
##     1.17651     0.32557
```

Yay! The p-value here is large. This suggests that we can actually drop all the interaction terms from the model. If you want to compare model summaries, you can. We only have a drop in robust R^2 of about 0.03, and we got rid of ALL the interaction terms. It's a much simpler model.

```
summary(eadmodreduced)
```

```
## Call:
## rfit.default(formula = EAD ~ AD.BD + AD.CD + EA.CD + Dx.CD +
##     SH.ACR + class, data = mythree)
##
## Coefficients:
##               Estimate Std. Error  t.value   p.value
## (Intercept) 196.379622  14.206206  13.8235 < 2.2e-16 ***
## AD.BD        -1.066442   0.127672  -8.3530 5.562e-12 ***
## AD.CD         0.036006   0.061815   0.5825  0.562204
## EA.CD        -1.066449   0.086240 -12.3661 < 2.2e-16 ***
## Dx.CD         2.716938   0.888080   3.0593  0.003188 **
## SH.ACR        0.060771   0.066892   0.9085  0.366879
## classPan      3.739660   3.538848   1.0567  0.294423
## classPongo    8.803808   1.642217   5.3609 1.101e-06 ***
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.8058012
## Reduction in Dispersion Test: 39.71533 p-value: 0
```

(If time permits) Find the "best" nonparametric model you can to predict beta, using any combination of predictors, interactions, re-expressed predictors, etc. for your data set with only 3 genera. Remember, we like simple models for ease of interpretation. (If you don't care about interpreting, then you can make a model more complicated if it improved prediction.) Your response should use at least 2 nonparametric test procedures (though they don't need to be drop-tests), and be sure you fit nonparametric models. Feel free to make comparisons to parametric methods.

SOLUTION:

Answers will vary. The idea is to explore using the drop.tests for subsets of predictors, and individual tests for individual terms in the presence of the others. The overall.test could be used to test for whether any term is useful.

Example 3 - Simulated Data

Here, we consider a setup with simulated data with the SAME underlying linear model but different distributions for the error terms. We generated the data separately and have uploaded the data sets. We are not sharing the linear model with you - see if you can figure it out while exploring the problems below.

The setting is as follows: The company WidgetsRUs sells widgets. They have three stores in a particular city, and they are trying to develop a model to predict the daily number of widgets sold based on some other variables: price of the widgets that day (dollars), whether there was a publicized sale on widgets that day (yes/no), and the location of the store (3 locations). They have provided data for you from the past 100 days of sales, which you can assume is a representative sample of their sales.

Normal Errors

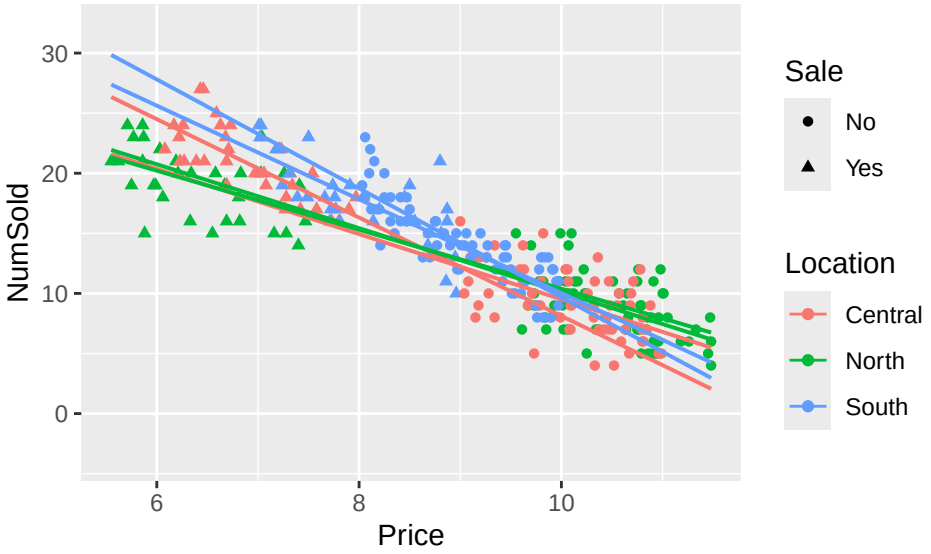
```
WidgetNormal<-
  read.csv("https://awagaman.people.amherst.edu/nonparametricstats/WidgetDataNormal.csv",
           h = T)
str(WidgetNormal)
```

```
## 'data.frame':    300 obs. of  4 variables:
##  $ NumSold : int  23 19 19 21 19 18 15 19 20 14 ...
##  $ Price   : num  5.87 5.99 6.67 6.19 5.97 6.81 7.28 6.69 7.26 7.4 ...
##  $ Sale    : chr  "Yes" "Yes" "Yes" "Yes" ...
##  $ Location: chr  "North" "North" "North" "North" ...
```

Find a "good" parametric regression line for predicting the daily NumSold values based on the other predictors. (No, you do not need to use all the predictors).

SOLUTION:

```
# plot parametric solutions to explore
ggplot(WidgetNormal, aes(x = Price, y = NumSold, color = Location, shape = Sale)) +
  geom_point() + geom_lm()
```




```
# fit the largest model we can - it has a three way interaction!
parafull <- lm(NumSold ~ Price*Location*Sale, data = WidgetNormal)
msummary(parafull)
```

```
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                  36.63624    4.64364   7.890 6.36e-14 ***
## Price                        -2.71241    0.45875  -5.913 9.51e-09 ***
## LocationNorth                 0.06622    6.93666   0.010  0.99239
## LocationSouth                18.37174    6.01421   3.055  0.00246 **
## SaleYes                      12.43047    6.65492   1.868  0.06280 .
## Price:LocationNorth           0.05322    0.67165   0.079  0.93690
## Price:LocationSouth          -1.82081    0.62455  -2.915  0.00383 **
## Price:SaleYes                -1.38173    0.82633  -1.672  0.09559 .
## LocationNorth:SaleYes       -14.17600    9.37917  -1.511  0.13177
## LocationSouth:SaleYes       -18.41624    8.97037  -2.053  0.04097 *
## Price:LocationNorth:SaleYes   1.58381    1.15432   1.372  0.17111
## Price:LocationSouth:SaleYes   2.01587    1.09441   1.842  0.06651 .
##
## Residual standard error: 2.148 on 288 degrees of freedom
## Multiple R-squared:  0.8479, Adjusted R-squared:  0.8421
## F-statistic:   146 on 11 and 288 DF,  p-value: < 2.2e-16
```

```
# the three-way interaction is "scary"; maybe we don't need it
para2way <- lm(NumSold ~ Price*Location + Price*Sale + Location*Sale,
               data = WidgetNormal)
anova(para2way, parafull)
```

```
## Analysis of Variance Table
##
## Model 1: NumSold ~ Price * Location + Price * Sale + Location * Sale
## Model 2: NumSold ~ Price * Location * Sale
##   Res.Df    RSS Df Sum of Sq      F Pr(>F)
## 1    290 1344.9
## 2    288 1328.3  2    16.572 1.7965 0.1677
```

```
msummary(para2way) # several other interactions aren't significant
```

```
##                         Estimate Std. Error t value Pr(>|t|)
## (Intercept)             40.63716    4.12227   9.858  < 2e-16 ***
## Price                   -3.10830    0.40706  -7.636 3.27e-13 ***
## LocationNorth           -5.09791    5.65475  -0.902   0.3681
## LocationSouth           12.07547    4.96075   2.434   0.0155 *
## SaleYes                  2.28521    3.79251   0.603   0.5473
## Price:LocationNorth      0.55999    0.54728   1.023   0.3071
## Price:LocationSouth     -1.16979    0.51413  -2.275   0.0236 *
## Price:SaleYes           -0.09723    0.45088  -0.216   0.8294
## LocationNorth:SaleYes   -1.65312    2.09873  -0.788   0.4315
## LocationSouth:SaleYes   -2.16798    1.43848  -1.507   0.1329
##
## Residual standard error: 2.153 on 290 degrees of freedom
## Multiple R-squared:  0.846,  Adjusted R-squared:  0.8412
## F-statistic:   177 on 9 and 290 DF,  p-value: < 2.2e-16
```

```
# we will try dropping some

# tried this model next
paralm <- lm(NumSold ~ Price*Location + Sale, data = WidgetNormal)
anova(paralm, para2way)
```

```
## Analysis of Variance Table
##
## Model 1: NumSold ~ Price * Location + Sale
## Model 2: NumSold ~ Price * Location + Price * Sale + Location * Sale
##   Res.Df    RSS Df Sum of Sq      F Pr(>F)
## 1    293 1356.3
## 2    290 1344.9  3    11.454 0.8233 0.4819
```

```
# sale isn't significant so take it out (overwrote model)
msummary(paralm)
```

```
##                       Estimate Std. Error t value Pr(>|t|)
## (Intercept)            46.1114     1.8703  24.654  < 2e-16 ***
## Price                  -3.6426     0.1918 -18.988  < 2e-16 ***
## LocationNorth          -9.7628     1.6118  -6.057 4.25e-09 ***
## LocationSouth           5.6133     2.7037   2.076   0.0388 *
## SaleYes                -0.3188     0.4990  -0.639   0.5234
## Price:LocationNorth     1.0176     0.1742   5.840 1.39e-08 ***
## Price:LocationSouth    -0.5345     0.3081  -1.735   0.0838 .
##
## Residual standard error: 2.152 on 293 degrees of freedom
## Multiple R-squared:  0.8447, Adjusted R-squared:  0.8415
## F-statistic: 265.6 on 6 and 293 DF,  p-value: < 2.2e-16
```

```
paralm <- lm(NumSold ~ Price*Location, data = WidgetNormal)
msummary(paralm)
```

```
##                       Estimate Std. Error t value Pr(>|t|)
## (Intercept)            45.2135     1.2328  36.674  < 2e-16 ***
## Price                  -3.5550     0.1341 -26.511  < 2e-16 ***
## LocationNorth          -9.6356     1.5978  -6.030  4.9e-09 ***
## LocationSouth           5.4131     2.6828   2.018   0.0445 *
## Price:LocationNorth     1.0027     0.1725   5.813  1.6e-08 ***
## Price:LocationSouth    -0.5075     0.3049  -1.665   0.0971 .
```

```
##
## Residual standard error: 2.149 on 294 degrees of freedom
## Multiple R-squared:  0.8445, Adjusted R-squared:  0.8419
## F-statistic: 319.3 on 5 and 294 DF,  p-value: < 2.2e-16
```

```
# could we really drop all that?
anova(paralm, parafull)
```

```
## Analysis of Variance Table
##
## Model 1: NumSold ~ Price * Location
## Model 2: NumSold ~ Price * Location * Sale
##   Res.Df    RSS Df Sum of Sq     F Pr(>F)
## 1    294 1358.2
## 2    288 1328.3  6    29.916 1.081 0.3738
```

After following a process to try to simplify the model, we ended up with a reduced model with just *Price* interacted with *Location*. The nested F-test suggests that we can go with the smaller model, which has an R^2 of 0.8445, which is not much loss (at all, compared to 0.8479 for the full model). In fact, the adjusted R^2 values are almost the same, 0.8421 for the full model and 0.8419 for the smaller model. Thus, we will use the smaller parametric model here.

Fit a nonparametric competitor using the same model structure.

SOLUTION:

```
nplm <- rfit(NumSold ~ Price*Location, data = WidgetNormal)
summary(nplm, overall.test = 'drop')
```

```
## Call:
## rfit.default(formula = NumSold ~ Price * Location, data = WidgetNormal)
##
## Coefficients:
##                     Estimate Std. Error  t.value   p.value
## (Intercept)         44.93825    1.26748  35.4548 < 2.2e-16 ***
## Price               -3.52941    0.13752 -25.6640 < 2.2e-16 ***
## LocationNorth       -9.14590    1.63870  -5.5812 5.428e-08 ***
## LocationSouth        5.21084    2.75139   1.8939   0.05922 .
## Price:LocationNorth  0.94725    0.17691   5.3544 1.733e-07 ***
## Price:LocationSouth -0.48253    0.31269  -1.5431   0.12387
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.7521741
## Reduction in Dispersion Test: 178.4633 p-value: 0
```

How do the models compare in terms of coefficient estimates? In terms of model fit? In terms of satisfying conditions? (You can get the residuals from the nonparametric models and make plots manually.)

SOLUTION:

```
paralm$coeff
```

```
##         (Intercept)               Price       LocationNorth       LocationSouth
##          45.2135198          -3.5550116          -9.6355655           5.4130606
## Price:LocationNorth Price:LocationSouth
##           1.0026710          -0.5075049
```

```
nplm$coeff
```

```
##          (Intercept)                Price       LocationNorth       LocationSouth
##            44.9382531           -3.5294112           -9.1459031            5.2108387
## Price:LocationNorth Price:LocationSouth
##            0.9472516           -0.4825310
```

The coefficients look to be very similar.

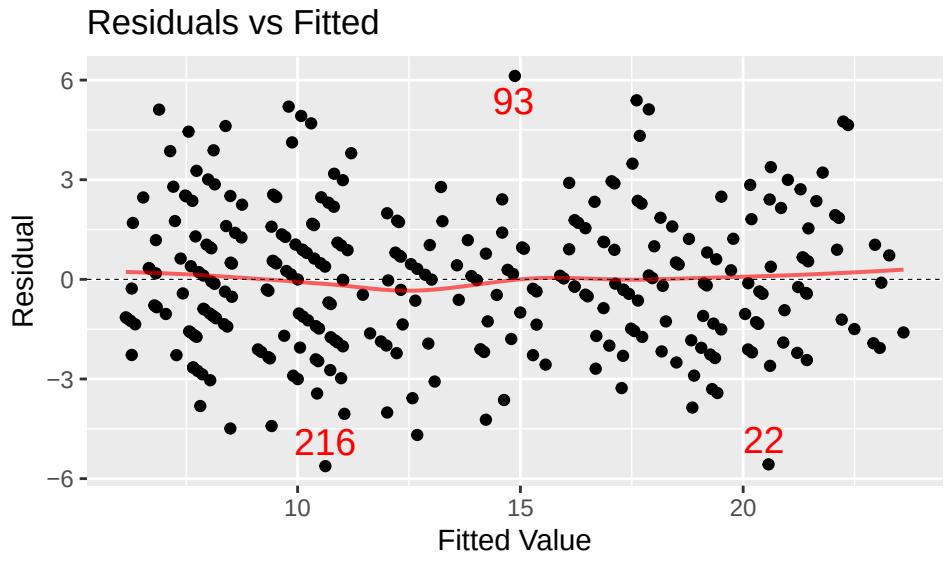
The parametric R^2 is 0.8445 and the nonparametric robust R^2 is 0.752. While the nonparametric R^2 is a little lower than the parametric R^2, both indicate a very good fit.

For conditions, we make plots! We assume the linear model is appropriate, that we have a random sample of errors which are independent from each other, and with mean/median 0.

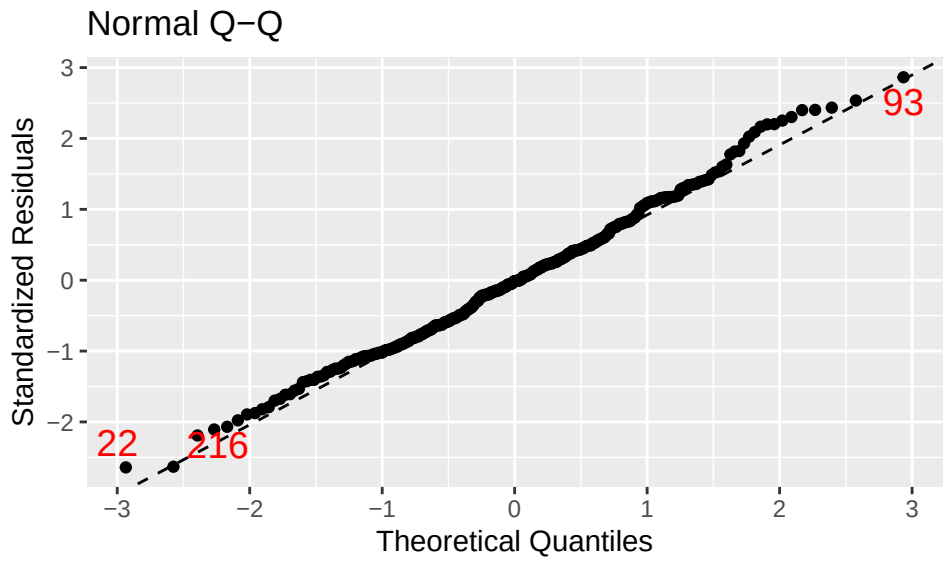
For the parametric model:

```
mplot(paralm, which = 1)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



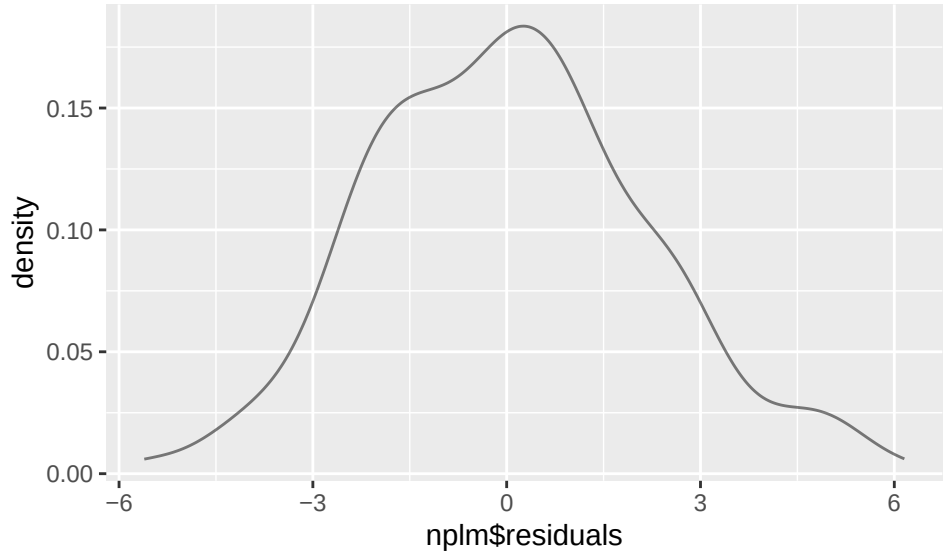

```
mplot(paralm, which = 2)
```



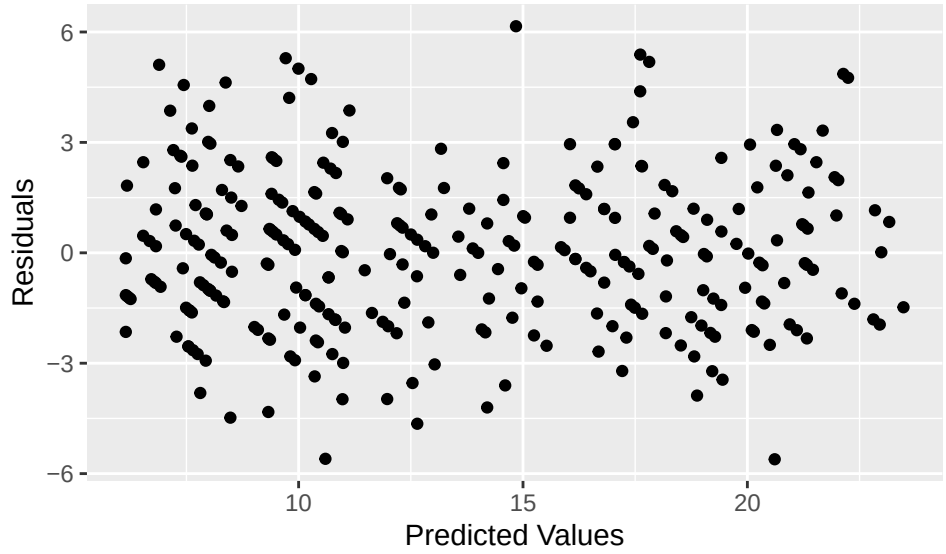
The constant variance condition seems reasonable, though the variance may decrease slightly in the middle and towards the end. A few outliers are labeled. The QQ plot seems to indicate reasonable normality as well, a minor deviation in the upper tail corrects itself. This may be due to the discrete nature of the response variable.

For the nonparametric model, we have the residuals, but the plot functionality is not automated. We can make the plots manually though. We need to check for similar spread still, and instead of normality, just need a continuous population.

```
gf_dens(~ nplm$residuals) # looks somewhat normal
```




```
# make residuals versus fitted plot manually
gf_point(residuals(nplm) ~ fitted(nplm),
         xlab = "Predicted Values", ylab = "Residuals")
```

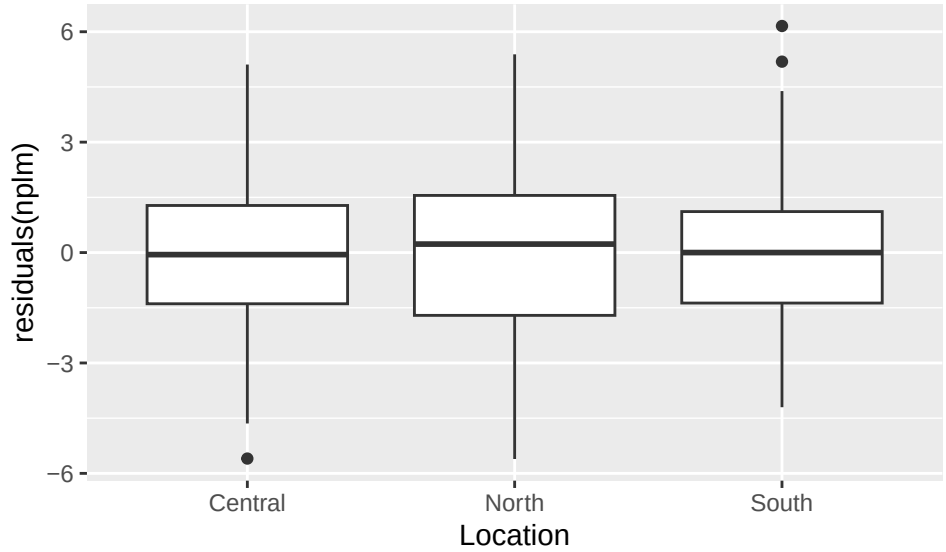


The constant variance condition is the one that likely still has the concern. We can check across the locations to see if there is a huge difference in spread. (Like you would check for an ANOVA).

```
favstats(residuals(nplm) ~ Location, data = WidgetNormal)
```

```
##   Location       min        Q1     median       Q3      max       mean       sd
## 1  Central -5.597082 -1.391200 -0.0559055 1.279388 5.108799 0.04350633 2.179212
## 2    North -5.609251 -1.706669  0.2299504 1.552720 5.386054 0.05861236 2.307349
## 3    South -4.202090 -1.371851  0.0000000 1.113314 6.155999 0.03920150 1.911338
##     n missing
## 1 100       0
## 2 100       0
## 3 100       0
```

```
gf_boxplot(residuals(nplm) ~ Location, data = WidgetNormal)
```



The spreads do not seem extremely different by location at least. (Note that since we generated them from the same population, technically, the simulation was set up with the condition met.)

Double Exp Errors

```
WidgetDExp <-
  read.csv("https://awagaman.people.amherst.edu/nonparametricstats/WidgetDataDExp.csv",
                        h = T)
```

Find a "good" parametric regression line for predicting the daily NumSold values based on the other predictors. (No, you do not need to use all the predictors).

SOLUTION:

```
parafull <- lm(NumSold ~ Price*Location*Sale, data = WidgetDExp)
msummary(parafull)
```

```
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                  36.32912    8.28243   4.386 1.62e-05 ***
## Price                        -2.62147    0.81822  -3.204  0.00151 **
## LocationNorth                 2.20673   12.37227   0.178  0.85856
## LocationSouth                11.98426   10.72698   1.117  0.26484
## SaleYes                      -7.62729   11.86975  -0.643  0.52101
## Price:LocationNorth          -0.20537    1.19795  -0.171  0.86400
## Price:LocationSouth          -1.15104    1.11396  -1.033  0.30233
## Price:SaleYes                 1.36737    1.47385   0.928  0.35431
## LocationNorth:SaleYes         0.04949   16.72874   0.003  0.99764
## LocationSouth:SaleYes         2.20545   15.99960   0.138  0.89046
## Price:LocationNorth:SaleYes -0.40538    2.05886  -0.197  0.84405
## Price:LocationSouth:SaleYes -0.85600    1.95200  -0.439  0.66133
##
## Residual standard error: 3.83 on 288 degrees of freedom
## Multiple R-squared:  0.5866, Adjusted R-squared:  0.5709
## F-statistic: 37.16 on 11 and 288 DF,  p-value: < 2.2e-16
```

```
# does the model from above still work?
paralm <- lm(NumSold ~ Price*Location, data = WidgetDExp)
msummary(paralm)
```

```
##                     Estimate Std. Error t value Pr(>|t|)
## (Intercept)          41.0060     2.1919  18.708   <2e-16 ***
## Price                -3.0715     0.2384 -12.883   <2e-16 ***
## LocationNorth        -6.0543     2.8409  -2.131   0.0339 *
## LocationSouth         1.7247     4.7699   0.362   0.7179
## Price:LocationNorth   0.5864     0.3067   1.912   0.0568 .
## Price:LocationSouth  -0.1076     0.5421  -0.199   0.8428
##
## Residual standard error: 3.821 on 294 degrees of freedom
## Multiple R-squared:   0.58,  Adjusted R-squared:  0.5729
## F-statistic:  81.2 on 5 and 294 DF,  p-value: < 2.2e-16
```

```
anova(paralm, parafull)
```

```
## Analysis of Variance Table
##
## Model 1: NumSold ~ Price * Location
## Model 2: NumSold ~ Price * Location * Sale
##   Res.Df    RSS Df Sum of Sq      F Pr(>F)
## 1    294 4293.5
## 2    288 4225.6  6    67.857 0.7708 0.5934
```

Following the same process as with the normal errors, we find a parametric nested F-test suggests that we can go with the smaller model with just *Price***Location*, which has an R^2 of 0.58, which is not much loss (at all) from the full R^2 of 0.5866. In fact, the smaller model has a higher adjust R^2 so we will use it here. We do note that this is a major drop in R^2 compared to when the errors are normal. The model just fits moderately well now.

Fit a nonparametric competitor using the same model structure.

SOLUTION:

```
nplm <- rfit(NumSold ~ Price*Location, data = WidgetDExp)
summary(nplm, overall.test = 'drop')
```

```
## Call:
## rfit.default(formula = NumSold ~ Price * Location, data = WidgetDExp)
##
## Coefficients:
##                      Estimate Std. Error  t.value   p.value
## (Intercept)         42.53618    1.94064  21.9186 < 2.2e-16 ***
## Price               -3.22154    0.21122 -15.2522 < 2.2e-16 ***
## LocationNorth       -7.89325    2.51681  -3.1362  0.001885 **
## LocationSouth        2.81062    4.22575   0.6651  0.506497
## Price:LocationNorth  0.78755    0.27171   2.8985  0.004032 **
## Price:LocationSouth -0.27008    0.48025  -0.5624  0.574292
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.5688975
## Reduction in Dispersion Test: 77.59448 p-value: 0
```

How do the models compare in terms of coefficient estimates? In terms of model fit? In terms of satisfying conditions? (You can get the residuals from the nonparametric models and make plots manually.)

SOLUTION:

```
paralm$coeff
```

```
##         (Intercept)               Price       LocationNorth       LocationSouth
##          41.0060363          -3.0714720          -6.0542707           1.7247357
## Price:LocationNorth Price:LocationSouth
##           0.5864340          -0.1076344
```

```
nplm$coeff
```

```
##         (Intercept)               Price       LocationNorth       LocationSouth
##          42.5361772          -3.2215357          -7.8932539           2.8106192
## Price:LocationNorth Price:LocationSouth
##           0.7875527          -0.2700805
```

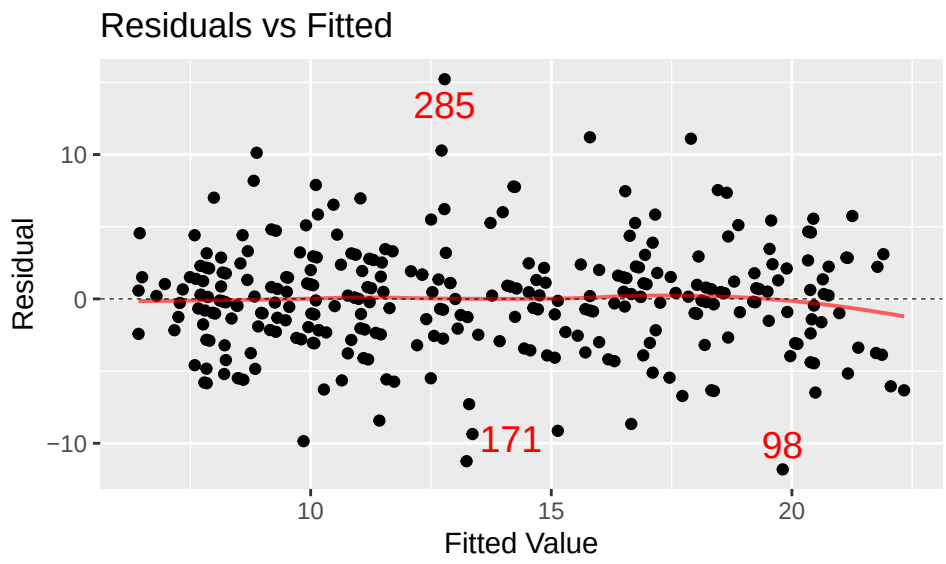
These coefficients are still very similar to each other.

The parametric R^2 is 0.58 and the nonparametric robust R^2 is 0.568. While the nonparametric R^2 is a little lower than the parametric R^2, it is closer than it was with the normal errors.

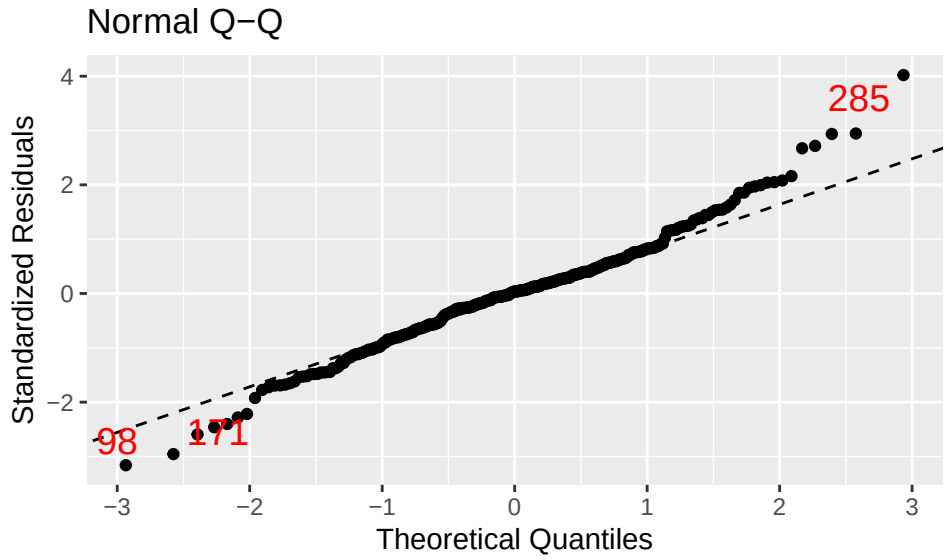
As before, for conditions, we make plots! We assume the linear model is appropriate, that we have a random sample of errors which are independent from each other, and with mean/median 0. For the parametric model:

```
mplot(paralm, which = 1)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



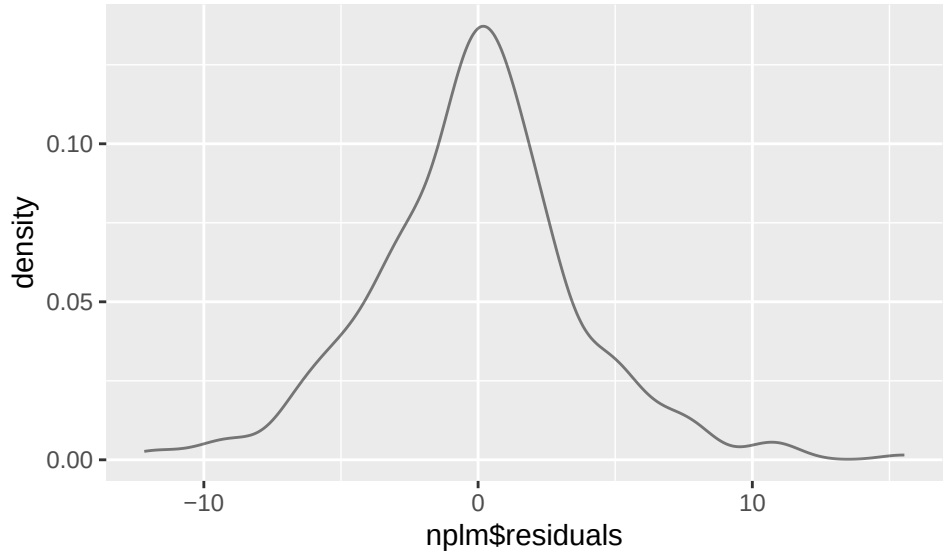

```
mplot(paralm, which = 2)
```



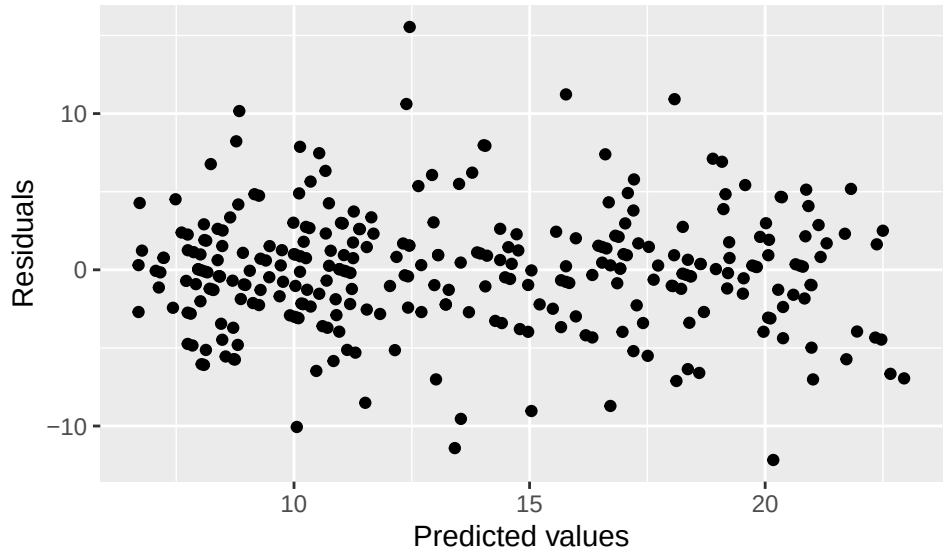
The constant variance condition seems reasonable, better than before, in fact. The QQ plot seems to indicate minor issues with normality with deviations from linear trend in both tails. The length of each tail (it's not just like 3 points) suggests this is likely serious enough to say the condition is not met. We could try a re-expression to see if that helps, if desired.

For the nonparametric model, we make the plots manually. We need to check for similar spread still, and instead of normality, just need a continuous population.

```
# looks bell-shaped, but likely has heavier tails than a normal
gf_dens(~ nplm$residuals)
```




```
gf_point(residuals(nplm) ~ fitted(nplm),
         xlab = "Predicted values", ylab = "Residuals")
```

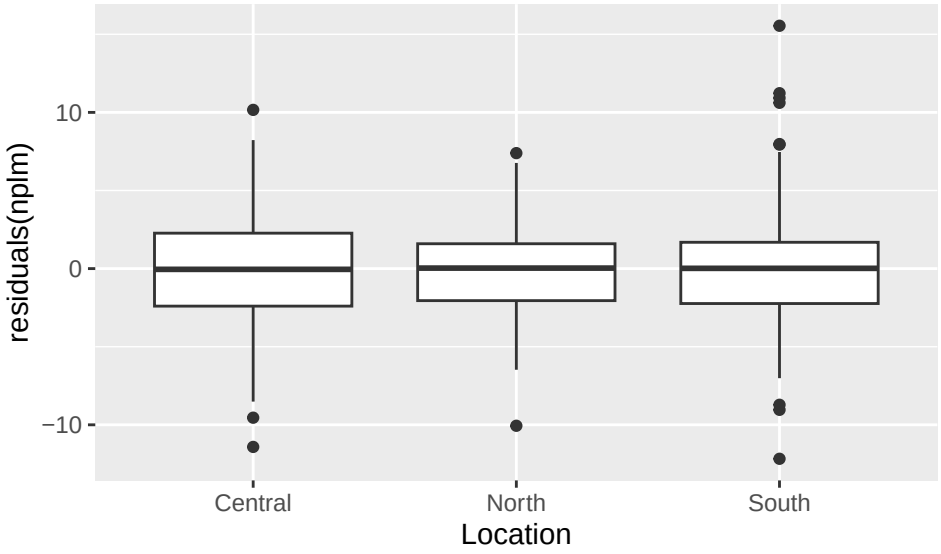


The constant variance condition is reasonable (the decrease in the middle is due to the sale/no sale transition, perhaps). We can check across the locations to see if there is a huge difference in spread. (Like you would check for an ANOVA).

```
favstats(residuals(nplm) ~ Location, data = WidgetDExp)
```

```
##   Location       min        Q1      median       Q3       max        mean
## 1  Central -11.41349 -2.401028 -0.04765200 2.269096 10.161086 -0.17161483
## 2    North -10.05970 -2.050626  0.03353023 1.591111  7.392891 -0.15859207
## 3    South -12.17224 -2.236497  0.01769536 1.685640 15.544228  0.09224742
##         sd   n missing
## 1 3.892568 100       0
## 2 3.266234 100       0
## 3 4.204472 100       0
```

```
gf_boxplot(residuals(nplm) ~ Location, data = WidgetDExp)
```



The SDs of the residuals across *Location* differ a bit more than we might like. However, (4.2/3.27) is max/min ratio of SDs, and that's below 2, and the IQRs in the boxplot are similar. (Note that since we generated them from the same population, technically, the simulation was set up with the condition met.)

Are you able to find a nonparametric competitor (with different model structure) that does better than this in terms of model fit or in terms of satisfying conditions?

SOLUTION:

```
nplm2 <- rfit(NumSold ~ Price*Location*Sale, data = WidgetDExp)
summary(nplm2, overall.test = 'drop')
```

```
## Call:
## rfit.default(formula = NumSold ~ Price * Location * Sale, data = WidgetDExp)
##
## Coefficients:
##                              Estimate Std. Error t.value   p.value
## (Intercept)                  43.54268    7.36227  5.9143 9.428e-09 ***
## Price                        -3.32790    0.72733 -4.5755 7.070e-06 ***
## LocationNorth                -4.26375   10.99792 -0.3877    0.6985
## LocationSouth                 8.07341    9.53539  0.8467    0.3979
## SaleYes                     -14.40797   10.55122 -1.3655    0.1732
## Price:LocationNorth           0.45181    1.06488  0.4243    0.6717
## Price:LocationSouth          -0.83877    0.99022 -0.8471    0.3977
## Price:SaleYes                 2.03370    1.31013  1.5523    0.1217
## LocationNorth:SaleYes         5.55075   14.87046  0.3733    0.7092
## LocationSouth:SaleYes         6.83029   14.22232  0.4803    0.6314
## Price:LocationNorth:SaleYes  -0.94611    1.83015 -0.5170    0.6056
## Price:LocationSouth:SaleYes  -1.25687    1.73516 -0.7244    0.4694
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.5757322
## Reduction in Dispersion Test: 35.52878 p-value: 0
```

The full model with three-way interactions only has a robust R^2 of 0.576. We are within 1% of that so that is pretty good in terms of R^2. Maybe we can improve in simplicity. Here, a lot of predictors are not significant given the presence of others in the model. Let's see how the nonparametric SLR does.

```
nplm3 <- rfit(NumSold ~ Price, data = WidgetDExp)
summary(nplm3, overall.test = 'drop')
```

```
## Call:
## rfit.default(formula = NumSold ~ Price, data = WidgetDExp)
##
## Coefficients:
##             Estimate Std. Error t.value   p.value
## (Intercept)  39.1160     1.2290  31.827 < 2.2e-16 ***
## Price        -2.8446     0.1349 -21.087 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.5364281
## Reduction in Dispersion Test: 344.8345 p-value: 0
```

```
drop.test(nplm2, nplm) # can use just Price*Location
```

```
##
## Drop in Dispersion Test
## F-Statistic     p-value
##     0.84293     0.53763
```

```
drop.test(nplm, nplm3) # want to keep the interaction and/or Location
```

```
##
## Drop in Dispersion Test
## F-Statistic     p-value
##    4.553194    0.001393
```

It seems our model is pretty solid.

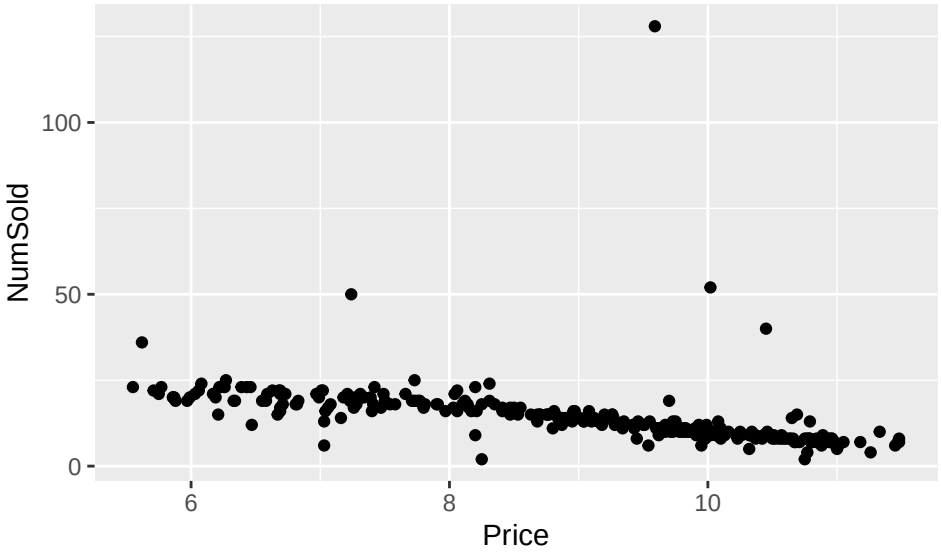
Cauchy Errors Note to avoid many negative *NumSold* values, we had to use a very small spread value for the Cauchy errors compared to the Normal and DExp cases. Two values were forced positive by redrawing errors.

```
WidgetCauchy <-
  read.csv("https://awagaman.people.amherst.edu/nonparametricstats/WidgetDataCauchy.csv",
                          h = T)
```

Find a "good" parametric regression line for predicting the daily *NumSold* values based on the other predictors. (No, you do not need to use all the predictors).

SOLUTION:

```
gf_point(NumSold ~ Price, data = WidgetCauchy)
```




```
parafull <- lm(NumSold ~ Price*Location*Sale, data = WidgetCauchy)
msummary(parafull)
```

```
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                    69.095     17.251   4.005 7.89e-05 ***
## Price                          -5.661      1.704  -3.322  0.00101 **
## LocationNorth                 -19.348     25.769  -0.751  0.45337
## LocationSouth                 -20.136     22.342  -0.901  0.36820
## SaleYes                       -32.350     24.723  -1.309  0.19174
## Price:LocationNorth             1.826      2.495   0.732  0.46498
## Price:LocationSouth             1.742      2.320   0.751  0.45328
## Price:SaleYes                   3.314      3.070   1.080  0.28124
## LocationNorth:SaleYes          28.798     34.843   0.827  0.40920
## LocationSouth:SaleYes          38.889     33.324   1.167  0.24419
## Price:LocationNorth:SaleYes    -3.689      4.288  -0.860  0.39042
## Price:LocationSouth:SaleYes    -4.021      4.066  -0.989  0.32352
##
## Residual standard error: 7.978 on 288 degrees of freedom
## Multiple R-squared:  0.2614, Adjusted R-squared:  0.2332
## F-statistic: 9.268 on 11 and 288 DF,  p-value: 3.129e-14
```

```
paralm <- lm(NumSold ~ Price*Location, data = WidgetCauchy)
msummary(paralm)
```

```
##                     Estimate Std. Error t value Pr(>|t|)
## (Intercept)          41.3806     4.5615   9.072  < 2e-16 ***
## Price                -2.9438     0.4962  -5.933 8.34e-09 ***
## LocationNorth        -5.9559     5.9121  -1.007    0.315
## LocationSouth        12.0064     9.9264   1.210    0.227
## Price:LocationNorth   0.4574     0.6383   0.717    0.474
## Price:LocationSouth  -1.4522     1.1281  -1.287    0.199
##
## Residual standard error: 7.953 on 294 degrees of freedom
## Multiple R-squared:  0.2509, Adjusted R-squared:  0.2381
## F-statistic: 19.69 on 5 and 294 DF,  p-value: < 2.2e-16
```

```
anova(paralm, parafull)
```

```
## Analysis of Variance Table
##
## Model 1: NumSold ~ Price * Location
## Model 2: NumSold ~ Price * Location * Sale
##   Res.Df   RSS Df Sum of Sq      F Pr(>F)
## 1    294 18594
## 2    288 18332  6     262.5 0.6873   0.66
```

Following the same process as with the normal errors, we find a parametric nested F-test suggests that we can go with the smaller model with just *Price***Location*, which has an R^2 of 0.2509, which is not much loss from the full R^2 of 0.2614. Thus, we will use the smaller model here. We do note that this is a major drop in R^2 compared to when the errors are normal, or when the errors were double exponential. The model does not fit very well now. We can see why in the plot - there are extreme outliers.

Fit a nonparametric competitor using the same model structure.

SOLUTION:

```
nplm <- rfit(NumSold ~ Price*Location, data = WidgetCauchy)
summary(nplm, overall.test = 'drop')
```

```
## Call:
## rfit.default(formula = NumSold ~ Price * Location, data = WidgetCauchy)
##
## Coefficients:
##                       Estimate Std. Error  t.value   p.value
## (Intercept)         44.743078   0.604076  74.0686 < 2.2e-16 ***
## Price               -3.478290   0.065797 -52.8640 < 2.2e-16 ***
## LocationNorth       -9.639976   0.784019 -12.2956 < 2.2e-16 ***
## LocationSouth        5.395534   1.316376   4.0988 5.378e-05 ***
## Price:LocationNorth  0.967128   0.084641  11.4262 < 2.2e-16 ***
## Price:LocationSouth -0.541811   0.149605  -3.6216 0.0003447 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.8664944
## Reduction in Dispersion Test: 381.6311 p-value: 0
```

Tada! The Robust R-Squared here is 0.8665, which is MUCH higher than the parametric counterpart. (This is why we suggest removing outliers to assess impact on analysis with parametric procedures.)

How do the models compare in terms of coefficient estimates? In terms of model fit? In terms of satisfying conditions? (You can get the residuals from the nonparametric models and make plots manually.)

SOLUTION:

```
paralm$coeff
```

```
##         (Intercept)               Price       LocationNorth       LocationSouth
##          41.3805728          -2.9438388          -5.9559027          12.0064309
## Price:LocationNorth Price:LocationSouth
##           0.4573913          -1.4522060
```

```
nplm$coeff
```

```
##         (Intercept)               Price       LocationNorth       LocationSouth
##          44.7430776          -3.4782896          -9.6399759           5.3955336
```

```
## Price:LocationNorth Price:LocationSouth
##           0.9671282          -0.5418109
```

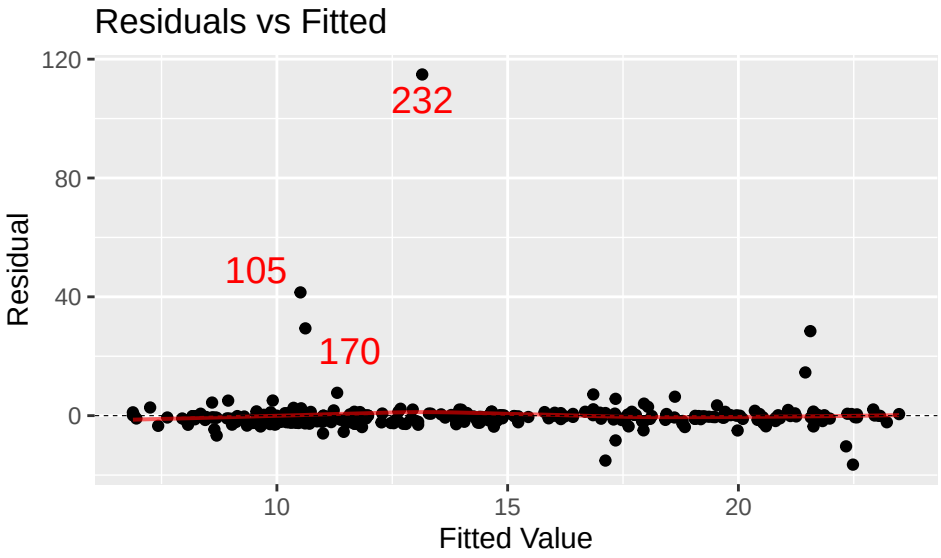
These coefficients are no longer similar. They have the same signs, but there is quite a difference in magnitude.

The parametric R^2 is 0.2509 and the nonparametric robust R^2 is 0.8665. The parametric model is a much worse fit than the nonparametric model here.

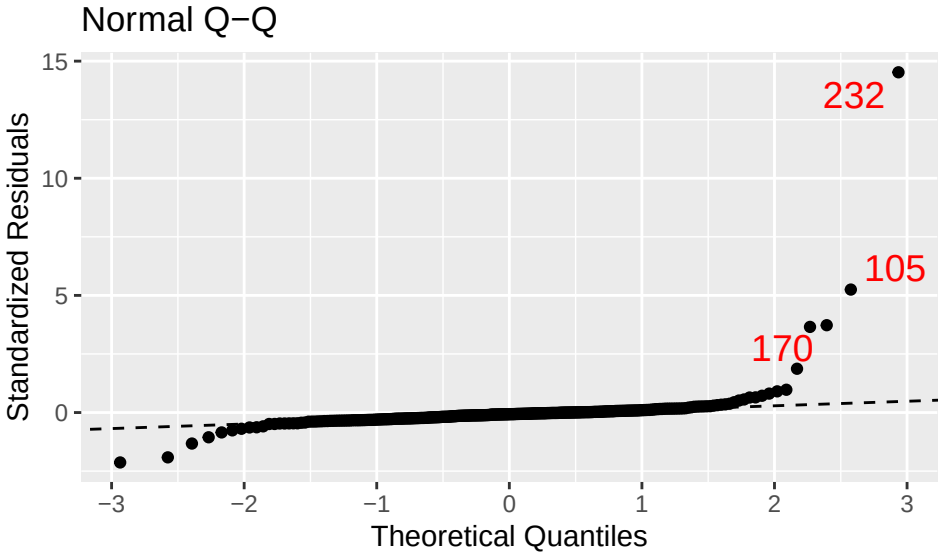
As before, for conditions, we make plots! We assume the linear model is appropriate, that we have a random sample of errors which are independent from each other, and with mean/median 0. For the parametric model:

```
mplot(paralm, which = 1)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



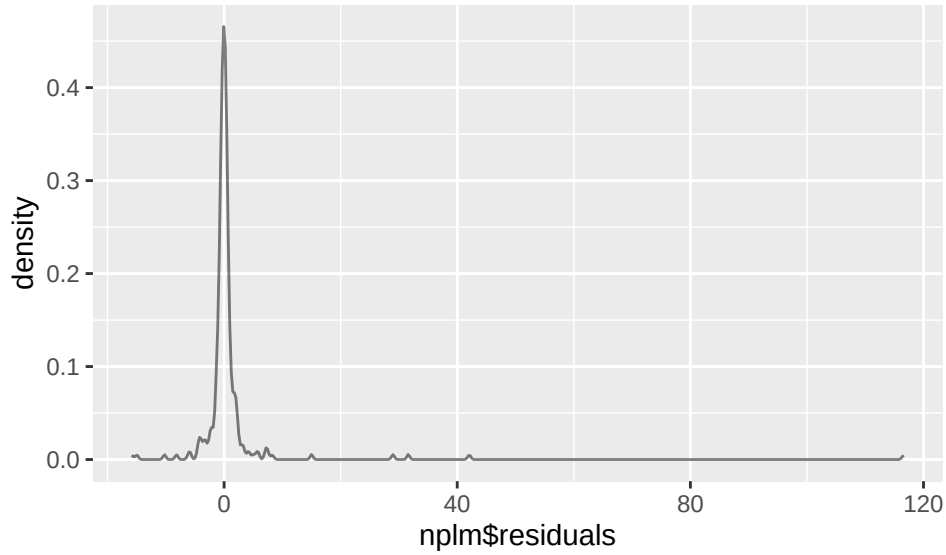

```
mplot(paralm, which = 2)
```



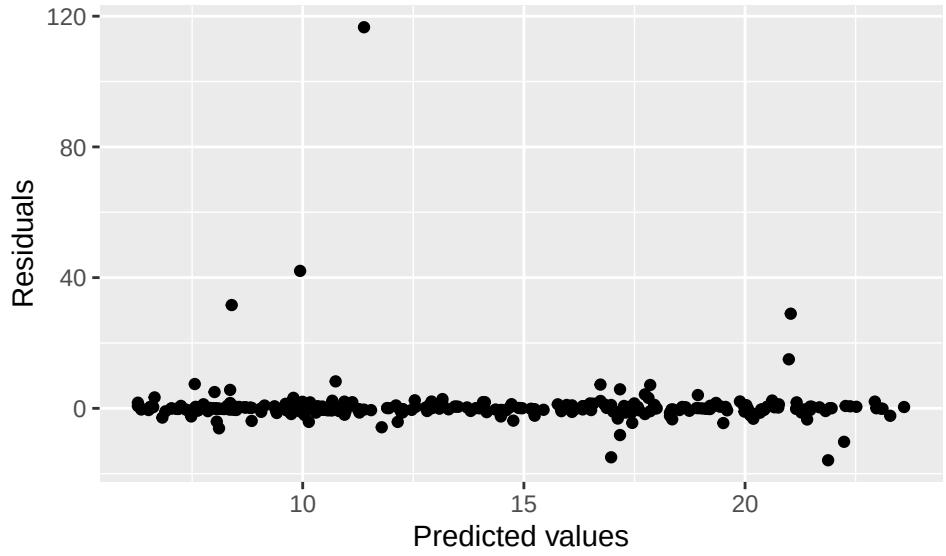
Nope! The outliers cause issues with constant variance and the heavy tails mean the distribution does not appear to be normal.

For the nonparametric model, we make the plots manually. We need to check for similar spread still, and instead of normality, just need a continuous population.

```
gf_dens(~ nplm$residuals) # one peak but has a long + tail
```




```
gf_point(residuals(nplm) ~ fitted(nplm),
         xlab = "Predicted values", ylab = "Residuals")
```



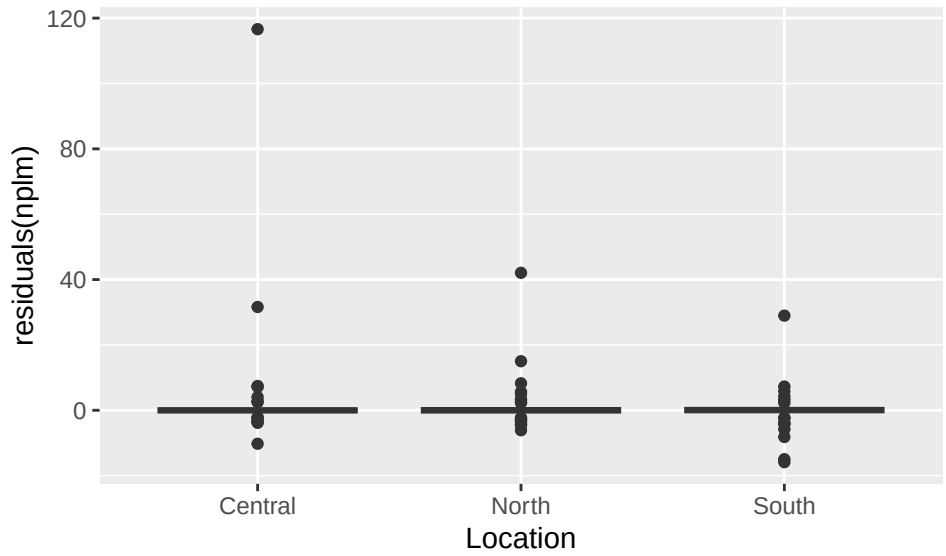
The constant variance condition has issues with the outliers, and the distribution of residuals has a long tail. Luckily, the outliers did not influence the regression line as much as they did in the parametric case. We can check across the locations to see if there is a huge difference in spread. (Like you would check for an ANOVA).

```
favstats(residuals(nplm) ~ Location, data = WidgetCauchy)
```

```
##   Location        min         Q1      median        Q3       max         mean
## 1  Central -10.238544 -0.5341486 -0.04276984 0.5071209 116.61372  1.475878555
## 2    North  -6.108116 -0.5657059  0.01046358 0.5764536  42.05874  0.547836959
```

```
## 3    South -15.877305 -0.4903701  0.01716760 0.6038510  28.96692 -0.009616316
##          sd   n missing
## 1 12.187348 100       0
## 2  4.837843 100       0
## 3  4.088207 100       0
```

```
gf_boxplot(residuals(nplm) ~ Location, data = WidgetCauchy)
```



The SDs differ a bit more than we might like due to the extreme outliers from the Cauchy, but the IQRs in the boxplot and seen in favstats are similar.

(Note that since we generated them from the same population, technically, the simulation was set up with the condition met.)

Are you able to find a nonparametric competitor (with different model structure) that does better than this in terms of model fit or in terms of satisfying conditions?

SOLUTION:

The results are very similar to what we found for the last type of errors.

```
nplm2 <- rfit(NumSold ~ Price*Location*Sale, data = WidgetCauchy)
summary(nplm2, overall.test = 'drop')
```

```
## Call:
## rfit.default(formula = NumSold ~ Price * Location * Sale, data = WidgetCauchy)
##
## Coefficients:
##                              Estimate Std. Error  t.value   p.value
## (Intercept)                 44.387270   2.323842  19.1008 < 2.2e-16 ***
## Price                       -3.442726   0.229589 -14.9951 < 2.2e-16 ***
## LocationNorth               -9.798486   3.471595  -2.8225  0.005097 **
## LocationSouth                5.922687   3.009934   1.9677  0.050060 .
## SaleYes                     -0.090166   3.330591  -0.0271  0.978421
## Price:LocationNorth          0.982582   0.336139   2.9231  0.003740 **
## Price:LocationSouth         -0.593728   0.312571  -1.8995  0.058498 .
## Price:SaleYes                0.033635   0.413555   0.0813  0.935236
## LocationNorth:SaleYes        3.988587   4.693998   0.8497  0.396186
## LocationSouth:SaleYes        0.378574   4.489405   0.0843  0.932856
```

```
## Price:LocationNorth:SaleYes -0.614337   0.577704  -1.0634  0.288486
## Price:LocationSouth:SaleYes -0.078814   0.547720  -0.1439  0.885685
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.8668172
## Reduction in Dispersion Test: 170.4038 p-value: 0
```

The full model with three-way interactions only has a robust R^2 of 0.8669. We are within 0.0004 of that so we are doing very well in terms of R^2. As before, maybe we can improve in simplicity. Here, a lot of predictors are not significant given the presence of others in the model. Let's see how the SLR does.

```
nplm3 <- rfit(NumSold ~ Price, data = WidgetCauchy)
summary(nplm3, overall.test = 'drop')
```

```
## Call:
## rfit.default(formula = NumSold ~ Price, data = WidgetCauchy)
##
## Coefficients:
##              Estimate Std. Error t.value   p.value
## (Intercept) 41.523386   0.574504  72.277 < 2.2e-16 ***
## Price       -3.118040   0.063516 -49.090 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.7772229
## Reduction in Dispersion Test: 1039.66 p-value: 0
```

```
drop.test(nplm2, nplm) # can use just Price*Location
```

```
##
## Drop in Dispersion Test
## F-Statistic     p-value
##     0.34911     0.91013
```

```
drop.test(nplm, nplm3) # want to keep the interaction and/or Location
```

```
##
## Drop in Dispersion Test
## F-Statistic     p-value
##      55.788       0.000
```

The SLR actually has a pretty big drop in terms of R^2. The drop in dispersion tests confirm our findings. It seems our model is pretty solid.

Model Comparison The underlying model structure did NOT change in any of the settings above. The errors changed in distribution.

Were the parametric coefficient estimates "stable"?

SOLUTION:

We saved over the names, so here are the three models and their coefficients (just the coeffs).

```
paralmN <- lm(NumSold ~ Price*Location, data = WidgetNormal)
paralmD <- lm(NumSold ~ Price*Location, data = WidgetDExp)
paralmC <- lm(NumSold ~ Price*Location, data = WidgetCauchy)
coefficients(paralmN)
```

```
##         (Intercept)               Price       LocationNorth       LocationSouth
##           45.2135198          -3.5550116          -9.6355655           5.4130606
## Price:LocationNorth Price:LocationSouth
##            1.0026710          -0.5075049
```

```
coefficients(paralmD)
```

```
##         (Intercept)               Price       LocationNorth       LocationSouth
##           41.0060363          -3.0714720          -6.0542707           1.7247357
## Price:LocationNorth Price:LocationSouth
##            0.5864340          -0.1076344
```

```
coefficients(paralmC)
```

```
##         (Intercept)               Price       LocationNorth       LocationSouth
##           41.3805728          -2.9438388          -5.9559027          12.0064309
## Price:LocationNorth Price:LocationSouth
##            0.4573913          -1.4522060
```

We see that the slope estimates are fairly different, and the location effects are also fairly radically changing. None of them change sign, but we'd definitely be reporting different lines with each of these models.

Were the nonparametric coefficient estimates "stable"?

SOLUTION:

We saved over the names, so here are the three models and their coefficients (just the coeffs).

```
nplmN <- rfit(NumSold ~ Price*Location, data = WidgetNormal)
nplmD <- rfit(NumSold ~ Price*Location, data = WidgetDExp)
nplmC <- rfit(NumSold ~ Price*Location, data = WidgetCauchy)
coefficients(nplmN)
```

```
##         (Intercept)               Price       LocationNorth       LocationSouth
##           44.9382531          -3.5294112          -9.1459031           5.2108387
## Price:LocationNorth Price:LocationSouth
##            0.9472516          -0.4825310
```

```
coefficients(nplmD)
```

```
##         (Intercept)               Price       LocationNorth       LocationSouth
##           42.5361772          -3.2215357          -7.8932539           2.8106192
## Price:LocationNorth Price:LocationSouth
##            0.7875527          -0.2700805
```

```
coefficients(nplmC)
```

```
##         (Intercept)               Price       LocationNorth       LocationSouth
##           44.7430776          -3.4782896          -9.6399759           5.3955336
## Price:LocationNorth Price:LocationSouth
##            0.9671282          -0.5418109
```

The DExp case seems to have thrown the nonparametric estimators a bit, though they aren't as far off as the parametric estimates in that setting. But the Cauchy and Normal nonparametric estimates are very close to each other.

Is there anything else you learned from these comparisons?

SOLUTION:

Answers will vary.

What we learned setting this up for you explore: Setting up simulated data can be tricky. But the Cauchy data illustrates what we wanted you to see very nicely.

Note that each of these data sets was a single draw of the errors from their corresponding distributions. To truly assess behavior here, doing this repeatedly would be advised.

Data Sources

Painted turtle data from Jolicoeur, Pierre, and James E. Mosimann. "Size and shape variation in the painted turtle. A principal component analysis." Growth 24.4 (1960): 339-354.

Primate data from Izenman, Alan J. Modern multivariate statistical techniques. Vol. 1. New York: Springer, 2008.

Original primate data source: Ashton, E. H., C. E. Oxnard, and T. F. Spence. "Scapular shape and primate classification." Proceedings of the Zoological Society of London. Vol. 145. No. 1. Oxford, UK: Blackwell Publishing Ltd, 1965.

The Widget data was simulated.

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to Nonparametric Statistical Methods (3rd Edition) by Hollander, Wolfe, and Chicken. Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

License